

AI WeatherWise

Intelligent Weather Forecasts & Contextual AI Insights (Backend Documentation)

1. PROJECT & TEAM REGISTRATION DETAILS

Project Title: AI WeatherWise – Intelligent Weather Forecasts & Insights

Domain: Full-Stack Web Development / Backend Architecture & Artificial Intelligence

Target Platform: Cloud-Ready Node.js REST API with Google Gemini AI Integration

TEAM LEADER

MOHAMED EZZATH A

Role: Project Architecture, API Integration & AI Logic

TEAM MEMBERS

1. DHANUSH M
2. HARISH KUMAR R
3. MARIA ANTONY A
4. VISHAL MADHAVAN M

2. WORK CONTRIBUTION BREAKDOWN MATRIX

This table records individual team member contributions and module allocations for final project evaluation.

S.No	Team Member Name	Assigned Role & Module
1	MOHAMED EZZATH A (Team Leader)	Project Architecture, API Integration & AI Logic
2	DHANUSH M	Backend Authentication & User Routes
3	HARISH KUMAR R	Weather API & Fallback Mock Service Integration
4	MARIA ANTONY A	Database Schemas & Location Management CRUD
5	VISHAL MADHAVAN M	API Endpoints Testing, Documentation & Validation

Document Purpose: This technical documentation serves as the comprehensive assessment report submitted under the Naan Mudhalvan scheme for final year evaluation.

3. EXECUTIVE PROJECT OVERVIEW

AI WeatherWise addresses the limitation of traditional weather applications by moving beyond static numerical forecasts. While standard applications provide temperature, humidity, and wind speed data, they leave the interpretation to the user. AI WeatherWise transforms raw weather parameters into actionable human insights using Generative Artificial Intelligence.

3.1 Problem Statement

Modern users frequently need to answer practical everyday questions: *"What should I wear today?"*, *"Is it suitable for an outdoor run?"*, *"Should I take an umbrella for my evening commute?"* Raw data alone requires mental processing. Our system automates this interpretation layer by feeding real-time meteorological metrics into an AI model tailored for lifestyle advice.

3.2 Core Objectives

- **RESTful API Architecture:** Build a high-performance backend using Express.js and Node.js.
- **Secure Access Management:** Implement stateless token-based authentication using JSON Web Tokens (JWT) and encrypted password storage with `bcryptjs`.
- **Real-time Data Fetching:** Integrate external meteorological APIs (OpenWeatherMap) with dynamic city queries.
- **Generative AI Integration:** Connect Google Gemini AI SDK (`@google/genai`) to process temperature, humidity, and atmospheric conditions, producing natural language summaries.
- **User-centric Data Persistence:** Allow registered users to maintain a persistent catalog of favorite cities stored in MongoDB.
- **High System Availability:** Implement fallback mock generation engines so that API rate limits or missing credentials never interrupt client application flow.

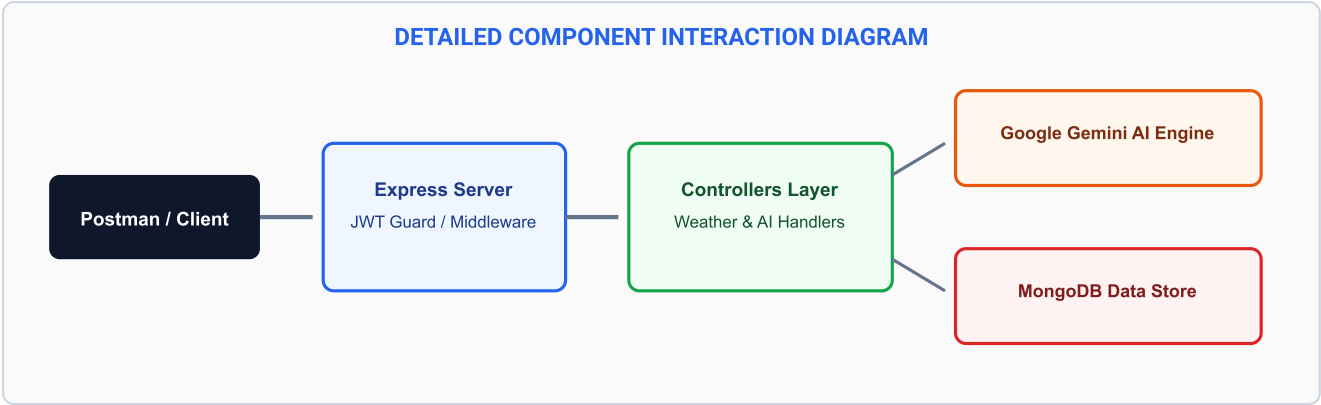
3.3 Functional Scope

SYSTEM SCOPE SUMMARY

The system acts as a central backend service serving web, mobile, or CLI clients. It strictly enforces boundary safety through middleware, validates inputs, queries persistent databases via Mongoose schemas, and communicates asynchronously with third-party web services.

4. SYSTEM ARCHITECTURE & WORKFLOW DESIGN

The backend is engineered around the Model-View-Controller (MVC) design pattern, separated into logical tiers to maximize maintainability, scalability, and code testability.



4.1 User Execution Flow

- 1. Authentication Request:** User registers/logs in via `/api/auth`. Upon validation, a signed JWT bearer token is returned to the client.
- 2. Protected Request Header:** Client attaches token as `Authorization: Bearer <TOKEN>` in subsequent API requests.
- 3. Middleware Verification:** Express middleware intersects incoming calls, decodes the token, verifies signatures, and appends the user context object.
- 4. Data Retrieval & Enrichment:** Weather controller calls OpenWeatherMap API. The resulting metric payload is simultaneously passed to Google Gemini AI to synthesize advice.
- 5. Database Persistence:** User-customized locations are queried or updated in MongoDB via Mongoose.

5. HARDWARE & SOFTWARE REQUIREMENTS

SOFTWARE STACK

- **Operating System:** Windows 10/11, macOS, Linux
- **Runtime Environment:** Node.js (v16.0.0 or higher)
- **Package Manager:** npm (v8.0.0 or higher)
- **Database Engine:** MongoDB (v5.0+ local or Atlas)
- **Primary Framework:** Express.js (v4.18+)
- **AI SDK:** [@google/genai](#)
- **API Testing:** Postman / Thunder Client

HARDWARE SPECIFICATIONS

- **Processor:** Intel Core i5 (8th Gen+) or AMD Ryzen 5
- **System RAM:** 8 GB minimum (16 GB recommended)
- **Storage Space:** 1 GB available storage
- **Network Adapter:** Active broadband connection
- **Display:** 1366x768 minimum resolution

6. DETAILED DIRECTORY STRUCTURE

The code layout enforces strict separation of concerns:

```
WEATHER/
├── src/
│   ├── config/
│   │   └── db.js           # MongoDB connection initialization
│   ├── controllers/
│   │   ├── aiController.js # Handles Gemini AI prompt construction
│   │   ├── authController.js # User register & login handlers
│   │   ├── locationController.js # CRUD handlers for user saved cities
│   │   └── weatherController.js # Weather fetch & fallback mock logic
│   ├── middleware/
│   │   └── authMiddleware.js # JWT authorization & route protector
│   ├── models/
│   │   ├── Location.js     # Schema for city bookmarking
│   │   └── User.js         # Schema for user credentials
│   ├── routes/
│   │   ├── aiRoutes.js     # Route endpoints for AI services
│   │   ├── authRoutes.js   # Auth routing endpoints
│   │   ├── locationRoutes.js # Location routing endpoints
│   │   └── weatherRoutes.js # Weather data endpoints
│   └── app.js              # Express app instantiation & routes configuration
├── .env                   # Environment key variables
├── package.json           # Dependencies manifest
└── index.js               # Server startup file listener
```

7. PROJECT SETUP & DATABASE SCHEMAS

7.1 Installation Steps

```
# 1. Initialize project directory
npm init -y

# 2. Install production dependencies
npm install express mongoose bcryptjs jsonwebtoken cors dotenv @google/genai

# 3. Install development scripts
npm install --save-dev nodemon
```

7.2 Database Configuration (`src/config/db.js`)

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI ||
'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

7.3 User Model Schema (`src/models/User.js`)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Please provide a name'], trim: true },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true
  },
  password: { type: String, required: [true, 'Please provide a password'], minlength: 6 }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

7.4 Saved Location Model Schema (`src/models/Location.js`)

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: { type: String, required: [true, 'Please provide a city name'], trim: true },
  country: { type: String, required: [true, 'Please provide a country name'], trim: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
}, { timestamps: true });

locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

module.exports = mongoose.model('Location', locationSchema);
```

8. CORE CONTROLLER LOGIC IMPLEMENTATION

8.1 Authentication Middleware (`src/middleware/authMiddleware.js`)

```
const jwt = require('jsonwebtoken');

const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret');
      req.user = decoded;
      return next();
    } catch (error) {
      return res.status(401).json({ success: false, message: 'Not authorized, token failed' });
    }
  }
  if (!token) {
    return res.status(401).json({ success: false, message: 'Not authorized, no token provided' });
  }
};

module.exports = { protect };
```

8.2 AI Recommendation Controller (`src/controllers/aiController.js`)

```
const { GoogleGenAI } = require('@google/genai');

const getWeatherRecommendation = async (req, res) => {
  const { city, temperature, humidity, condition } = req.body;
  try {
    const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
    const prompt = `Location: ${city}, Temp: ${temperature}°C, Humidity: ${humidity}%, Weather: ${condition}.
    Provide a concise 2-sentence recommendation for outfit and outdoor activities.`;

    const response = await ai.models.generateContent({
      model: 'gemini-2.5-flash',
      contents: prompt,
    });

    return res.status(200).json({
      success: true,
      recommendation: response.text
    });
  } catch (error) {
    return res.status(200).json({
      success: true,
      recommendation: 'Dress comfortably for current temperature and stay hydrated.',
      isFallback: true
    });
  }
};

module.exports = { getWeatherRecommendation };
```

9. API ENDPOINTS & POSTMAN VERIFICATION

Endpoint Path	Method	Access Tier	Functional Objective
/api/auth/register	POST	Public	Registers user and returns JWT bearer token.
/api/auth/login	POST	Public	Authenticates credentials and issues login session token.
/api/weather/:city	GET	Public / Private	Fetches live meteorological parameters or fallback mock.
/api/ai/weather-recommendation	POST	Private (JWT)	Generates AI activity and clothing recommendations.
/api/locations	GET/POST	Private (JWT)	Manages user's favorited cities database records.

9.1 Postman Output: Register Request

```
// POST /api/auth/register
{
  "success": true,
  "data": {
    "_id": "643014436c279d74f6a1102b",
    "name": "Mohamed Ezzath A",
    "email": "ezzath@example.com",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

9.2 Postman Output: Live Weather Fetch

```
// GET /api/weather/bangalore
{
  "success": true,
  "data": {
    "city": "Bangalore",
    "temperature": 34,
    "humidity": 79,
    "windSpeed": 12,
    "condition": "Windy",
    "isMock": false
  }
}
```

9.3 Postman Output: AI Generation Response

```
// POST /api/ai/weather-recommendation
{
  "success": true,
  "recommendation": "The high humidity of 79% combined with 34°C heat makes it feel quite warm; wear breathable cotton clothing and ensure you stay well-hydrated throughout the day."
}
```

Conclusion: AI WeatherWise successfully demonstrates the power of augmenting backend services with generative AI models, delivering clear user value beyond conventional data reporting.